

Тестовое задание .NET: Мини-PDM «Состав изделия»

Введение

Нужно разработать упрощённую систему учёта конструкторского состава изделия: импорт из «CAD-файлов», хранение в реляционной базе данных, расчёты по дереву состава и десктопный интерфейс.

В реальной работе вы будете интегрироваться с САПР КОМПАС-3D и PLM-системой ЛОЦМАН:PLM. Их SDK передать нельзя, поэтому в задании они заменены эмуляцией:

- **CAD-система** — набор JSON-файлов, описывающих документы сборок и деталей. Работа с ними должна быть спрятана за интерфейсом так, будто за ним стоит настоящий API САПР.
- **PLM-система** — ваша собственная упрощённая модель данных: объекты, версии, состояния и связи «входит в».

Срок: не более 7 календарных дней. Результат отправлять по адресу tdragan@ascon-ufa.ru.

Если что-то в задании неоднозначно, примите разумное решение и опишите его в README.

Как будет оцениваться результат выполнения задания

1. Результативность. Задание выполнено в срок и в полном объеме.
2. Общий подход к решению. Возможность построения общего решения, а не решения «в лоб».
3. Уровень использования готовых решений и библиотек.
4. Уровень и целесообразность применения ООП/ООД.
5. Стиль кодирования.

Технологии

Выбор технологий в основном свободный. Использование нашего стека будет плюсом при прочих равных.

Область	Используем мы	Допустимо в задании
Интерфейс	WPF, Avalonia	WPF или Avalonia на выбор
СУБД	PostgreSQL, MS SQL	Любая реляционная СУБД с рекурсивными CTE (в том числе SQLite); PostgreSQL или MS SQL — плюс
Доступ к данным	Dapper	Любой: Dapper, EF Core, ADO.NET
Внедрение зависимостей	Jab	Любой контейнер или ручная композиция
Тесты	xUnit	xUnit, NUnit или MSTest
Платформа	.NET Framework 4.8, .NET 8	.NET 8 или новее

Предметная область

Система хранит объекты трёх типов, их версии и связи состава «входит в».

Тип объекта	Обозначение	Масса	Материал
Сборка (Assembly)	Обязательно	Не задаётся, вычисляется по составу	Нет
Деталь (Part)	Обязательно	Обязательна, кг за 1 шт.	Обязателен
Стандартное изделие (StandardPart)	Отсутствует; изделие идентифицируется по наименованию	Обязательна, кг за 1 шт.	Нет

Формат обозначения (упрощённый ЕСКД): четыре заглавные кириллические буквы, точка, шесть цифр, точка, три цифры. Пример: АБВГ.301245.001. Обозначение уникально среди сборок и деталей; наименование уникально среди стандартных изделий.

Версия объекта имеет номер (1, 2, 3...) и состояние:

- «В работе» — версию можно изменять;
- «Утверждено» — версию изменять нельзя, изменения возможны только через новую версию;
- «Аннулировано» — версия не участвует в расчётах.

Переходы: «В работе» → «Утверждено», «В работе» → «Аннулировано», «Утверждено» → «Аннулировано». Обратных переходов нет.

Текущая версия объекта — последняя неаннулированная. Её атрибуты используются в расчётах и показываются в интерфейсе.

Связь «Состоит из ...» соединяет версию сборки-родителя с дочерним объектом (не с его версией) и хранит количество — целое число больше нуля. При расчётах берётся текущая версия дочернего объекта.

Повторный импорт объекта, который уже есть в базе:

- данные не изменились — ничего не происходит;
- текущая версия «В работе» — она обновляется;
- текущая версия «Утверждено» — создаётся новая версия «В работе».

Рекомендуемая схема данных

Схема приведена как ориентир. Её можно менять, если вы обоснуете решение в README.

Таблица	Назначение	Основные поля
pdm_object	Объект: сборка, деталь или стандартное изделие	id, object_type, designation (уникально, NULL у стандартных изделий), name, current_version_id
object_version	Версия объекта и её атрибуты	id, object_id, version_no (уникально в пределах объекта), state, material, mass_kg (decimal), created_at
bom_link	Связь состава «Состоит из ...»	id, parent_version_id, child_object_id, quantity (> 0); уникальная пара родитель + потомок
import_log	Журнал импорта (необязательно)	id, started_at, file_name, severity, reason

Поле `current_version_id` избыточно, но заметно упрощает рекурсивный запрос по дереву состава. Его нужно поддерживать при создании и аннулировании версий.

Входные данные

К заданию приложен архив `cad-export.zip` с двумя папками:

- `cad-export` — 45 файлов, которые эмулируют документы CAD-системы. Это основной набор.
- `cad-export-v2` — та же модель после доработки конструктором. Нужна для проверки повторного импорта и версий.

Файлы `*.a3d` — сборки, `*.m3d` — детали и стандартные изделия. Содержимое каждого файла — JSON в UTF-8, независимо от расширения. Идентификатор документа — имя файла на диске. Сборка ссылается на компоненты по имени файла в той же папке.

Примеры ниже взяты из приложенного набора. Сборка:

```
{
  "formatVersion": 1,
  "fileName": "Вал промежуточный в сборе.a3d",
  "type": "Assembly",
  "designation": "РДЦЛ.304112.300",
  "name": "Вал промежуточный в сборе",
  "properties": { "material": null, "mass": null },
  "components": [
    { "file": "Вал промежуточный.m3d", "count": 1 },
    { "file": "Колесо зубчатое.m3d", "count": 1 },
    { "file": "Подшипник 208 ГОСТ 8338-75.m3d", "count": 2 },
    { "file": "Шпонка 12x8x40 ГОСТ 23360-78.m3d", "count": 1 },
    { "file": "Кольцо А40 ГОСТ 13942-86.m3d", "count": 2 }
  ]
}
```

Деталь и стандартное изделие:

```
{
  "formatVersion": 1,
  "fileName": "Колесо зубчатое.m3d",
  "type": "Part",
  "designation": "РДЦЛ.304112.302",
  "name": "Колесо зубчатое",
  "properties": { "material": "Сталь 40X", "mass": 5.86 },
  "components": []
}

{
  "formatVersion": 1,
  "fileName": "Подшипник 208 ГОСТ 8338-75.m3d",
  "type": "StandardPart",
  "designation": null,
  "name": "Подшипник 208 ГОСТ 8338-75",
  "properties": { "material": null, "mass": 0.37 },
  "components": []
}
```

Масса указана в килограммах за одну штуку.

Ошибки в данных и отчёт об импорте

Набор намеренно содержит ошибочные данные. Какие именно — часть задания: их нужно обнаружить. Правила обработки:

- Файл с ошибкой не импортируется, но импорт остальных файлов продолжается.
- Если у нескольких файлов совпадает обозначение, отклоняются все такие файлы.

- Если у компонента сборки количество не больше нуля, отклоняется вся сборка.
- Сборка, ссылающаяся на отклонённый или отсутствующий объект, тоже отклоняется, и так далее вверх по дереву.
- Деталь без массы импортируется с предупреждением. Расчёт массы сборки, где она встречается, должен сообщить, у какого компонента нет массы, а не вернуть ноль или неполную сумму. В сводной спецификации такая строка показывается с пустой массой.

По итогам импорта формируется отчёт. Пример формата (данные вымышленные, в приложенном наборе их нет):

Файл	Результат	Причина
Узел выходного вала.a3d	Ошибка	Ссылка на отсутствующий файл «Муфта МУВП-32.m3d»
Привод конвейера.a3d	Ошибка	Компонент «Узел выходного вала.a3d» отклонён
Крышка смотровая.m3d	Предупреждение	Не указана масса

Кроме таблицы, в отчёте должно быть указано, сколько объектов принято, отклонено и получило предупреждение.

Обязательная часть

1. Абстракция над CAD-системой

Работа с файлами должна идти только через интерфейс. Представьте, что позже вместо JSON за ним окажется настоящий API САПР:

```
public interface ICadDocumentReader
{
    Task<CadDocument> ReadAsync(string path, CancellationToken ct);
}
```

Сигнатуру и модель CadDocument можно менять, если вы обоснуете это в README. Остальной код не должен знать о JSON и файловой системе.

2. Хранение данных

1. Одна СУБД на ваш выбор (см. раздел «Технологии»).
2. Схема создаётся скриптом или миграцией, которые лежат в репозитории.
3. Дерево состава изделия выбирается одним запросом с рекурсивным CTE, а не циклом запросов из C#.
4. Импорт папки выполняется в транзакции.

3. Бизнес-логика

1. Расчёт массы сборки с учётом вложенности и количеств.
2. Сводная спецификация сборки: плоский список всех деталей и стандартных изделий с суммарным количеством по всему дереву и массой. Если одно изделие встречается на разных уровнях, количества перемножаются по пути и складываются.
3. Обнаружение циклических ссылок.
4. Правила версий и состояний из раздела «Предметная область», включая повторный импорт.

4. Интерфейс: WPF или Avalonia

1. Архитектура MVVM; логики в code-behind нет.
2. Дерево состава изделия.
3. Карточка выбранного объекта: атрибуты, версия, состояние, смена состояния.
4. Поиск по обозначению или наименованию.
5. Импорт папки с показом отчёта после завершения.
6. Расчёт массы и просмотр сводной спецификации для выбранной сборки.

Ниже ориентир по составу экрана, а не макет для точного повторения. Компоновку и оформление можно выбрать свои.

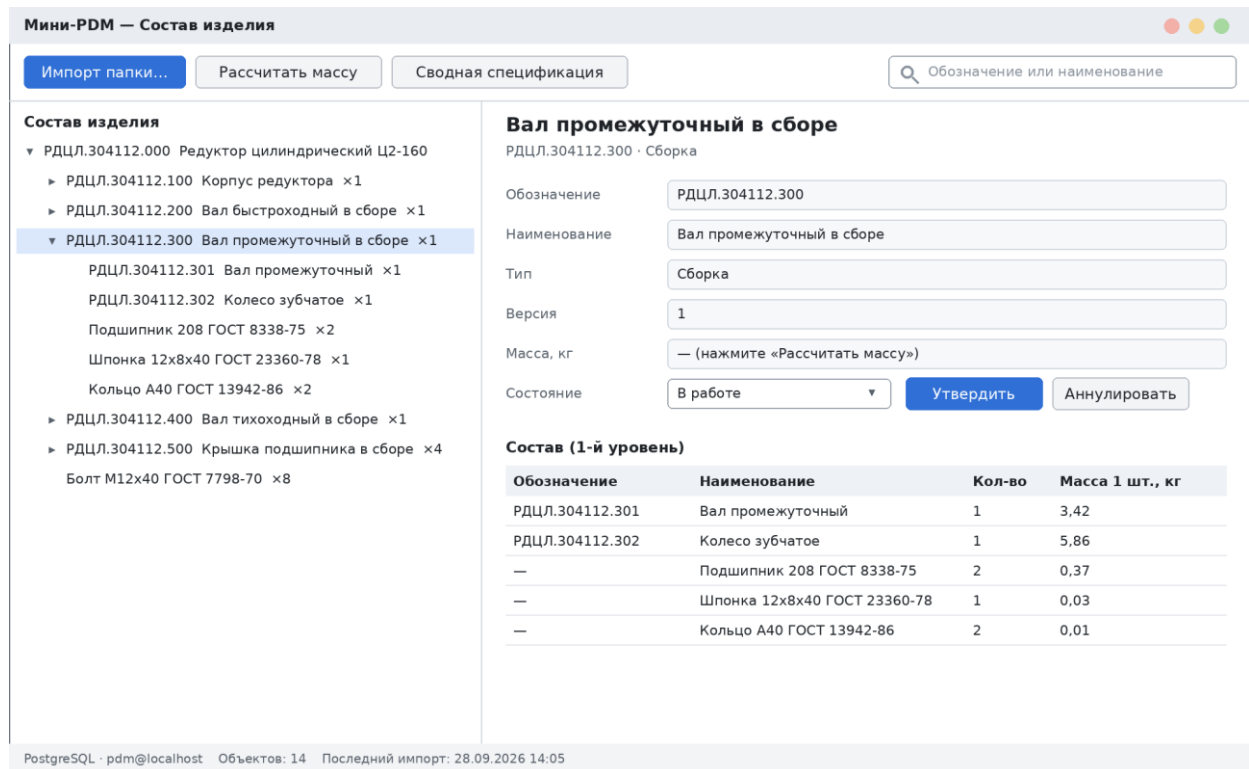


Рисунок 1. Главное окно: дерево состава, карточка объекта, состав первого уровня

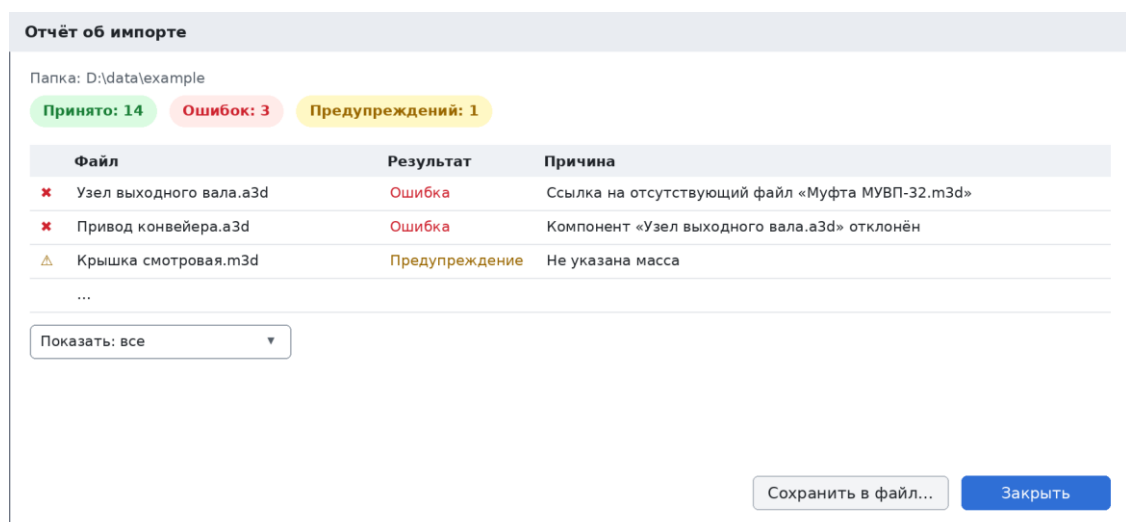


Рисунок 2. Отчёт об импорте (значения условные)

5. Композиция приложения

Зависимости собираются в одном месте любым способом: контейнером или вручную. Компоненты получают зависимости через конструктор.

6. Тесты

1. Расчёт массы, сводная спецификация, обнаружение циклов, валидация обозначений, правила перехода состояний.
2. Логика импорта тестируется с подменой `ICadDocumentReader`, без обращения к файловой системе.
3. Граничные случаи, а не только успешные сценарии.

Дополнительная часть

Не обязательно, но будет плюсом.

- Ленивая подгрузка дочерних узлов дерева состава.
- Асинхронный импорт с индикатором прогресса и возможностью отмены; интерфейс не зависает.
- Поддержка двух СУБД (PostgreSQL и MS SQL) с выбором провайдера в конфигурации; различия SQL-диалектов изолированы в одном месте.
- Использование нашего стека: Dapper, Jab, xUnit.
- `docker-compose.yml` для локального запуска СУБД.
- Экспорт сводной спецификации в CSV или XLSX.
- Сравнение двух версий сборки: что добавлено, удалено, где изменилось количество.
- Структурированное логирование, например Serilog.

Что сдать

1. Ссылку на Git-репозиторий (GitHub, GitLab или архив с папкой `.git`).
2. README, в котором описано:
 - как собрать и запустить приложение и тесты;
 - какая СУБД используется и где лежат скрипты схемы;
 - принятые архитектурные решения и их причины;
 - что не успели сделать и как бы это сделали;
 - отчёт импорта по набору `cad-export` и что изменилось после импорта `cad-export-v2` — скриншоты или текст.

Решение должно собираться командой `dotnet build` и тестироваться командой `dotnet test` без ручной настройки, кроме строки подключения к БД.